

OpenGL 셰이더 사용하기 2

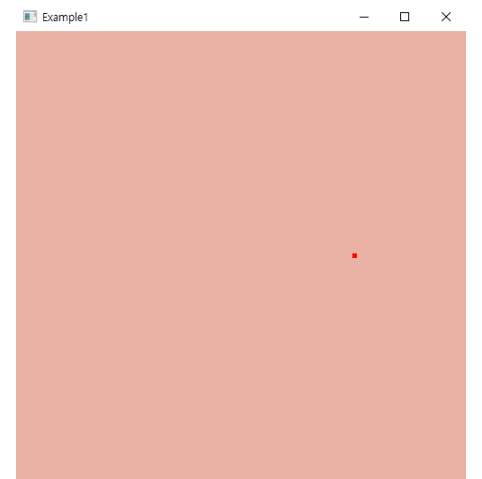
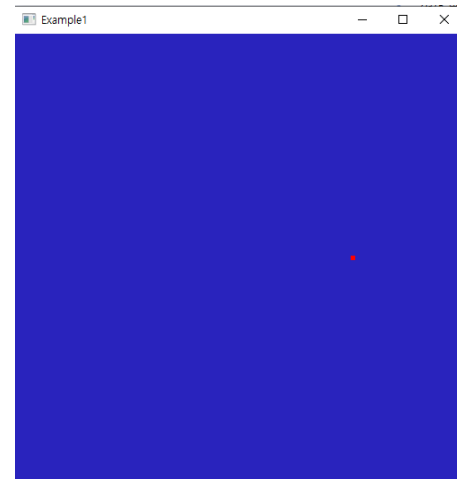
2026년 2학기

<셰이더 사용하여 도형 그리기>

1. 점 그리기
 - 셰이더에 고정된 한 점의 좌표값, 고정된 색상으로 점 그리기
2. 삼각형 그리기
 - 셰이더에 고정된 삼각형 좌표값, 고정된 색상으로 삼각형 그리기
3. 삼각형 그리기
 - 1) 메인 프로그램에서 좌표값을 정하여 버텍스 셰이더로 보내서 고정된 색상으로 삼각형 그리기 → Vertex Attribute 1개인 경우
 - 2) 좌표값과 색상을 메인프로그램에서 설정하고, 좌표값과 색상을 버텍스 셰이더로 보내고, 버텍스 셰이더에서 색상을 프래그먼트 셰이더로 보내서 삼각형 그리기 → Vertex Attribute 2개인 경우
 - 3) 좌표값과 색상을 각각 버텍스 셰이더와 프래그먼트 셰이더로 보내서 삼각형 그리기 → uniform 변수 사용하는 경우

1. 첫 번째 셰이더 프로그램: 화면에 점 찍기

- 화면에 점 찍기: 셰이더 내에 좌표값과 색상을 고정하여 사용하기
- 배경색을 칠하고 화면에 고정된 위치에 점 찍기
 - 버텍스 셰이더와 프래그먼트 셰이더는 각각 별개의 파일로 저장하여 사용
 - 버텍스 셰이더: vertex.glsl
 - 프래그먼트 셰이더: fragment.glsl
 - 파일의 확장자는 다르게 설정 가능 (우리는 glsl로 설정하기로 함)
 - 셰이더 코드 작성하기
 - 버텍스 셰이더: 점의 좌표값 설정 → (0.5, 0.0, 0.0)
 - 프래그먼트 셰이더: 고정 색상 설정 → (1.0, 0.0, 0.0)
 - 메인 프로그램 작성하기
 - 화면 띄우기
 - 버텍스 셰이더 객체와 프래그먼트 셰이더 객체 만들기
 - 셰이더 프로그램 만들어 두 셰이더 객체 연결하기
 - 출력 콜백 함수에서 그리기 함수 호출하여 점 찍기



셰이더 코드

- 버텍스 셰이더: vertex.glsl 파일로 저장

```
#version 330 core
```

```
void main()
```

```
{  
    gl_Position = vec4(0.5, 0.0, 0.0, 1.0);    //--- (0.5, 0.0, 0.0) 으로 좌표값 고정  
}
```

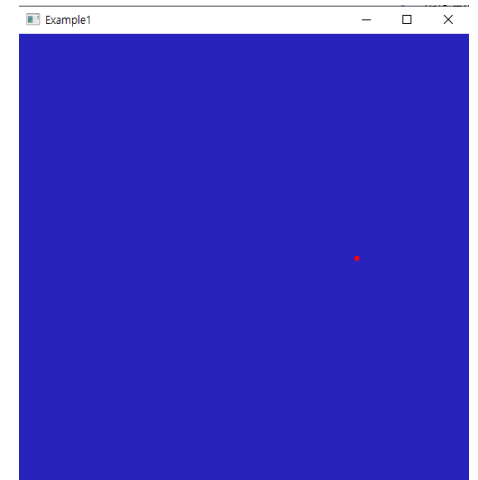
- 프래그먼트 셰이더: fragment.glsl 파일로 저장

```
#version 330 core
```

```
out vec4 color;
```

```
void main ()
```

```
{  
    color = vec4 (1.0, 0.0, 0.0, 1.0);    //--- 빨강색으로 색상 고정  
}
```



셰이더 사용하여 점 찍기: 메인 함수

//--- 필요한 헤더파일 선언

```
#include <GL/glew.h>
#include <GL/glfw3.h>
```

//--- 사용자 정의 함수

```
void make_vertexShaders();
void make_fragmentShaders();
GLuint make_shaderProgram();
GLvoid drawScene();
GLvoid Reshape(int w, int h);
```

//--- 필요한 변수 선언

```
GLint width, height;
GLuint shaderProgramID;
GLuint vertexShader;
GLuint fragmentShader;
```

//--- 메인 함수

```
int main(int argc, char** argv)
{
    width = 500;
    height = 500;
```

//--- GLFW 초기화

```
if (!glfwInit())
    return -1;
```

//--- OpenGL 버전 및 프로파일 설정

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

//--- 윈도우 생성

```
GLFWwindow* window =
    glfwCreateWindow(width, height, "Example1", nullptr, nullptr);
```

```
if (!window)
```

```
{
    glfwTerminate();
    return -1;
}
```

//--- 생성한 윈도우의 OpenGL Context를 현재 Context로 설정

```
glfwMakeContextCurrent(window);
```

//--- GLEW 초기화

```
glewExperimental = GL_TRUE;
if (glewInit() != GLEW_OK)
{
    glfwDestroyWindow(window);
    glfwTerminate();
    return -1;
}
```

//--- 셰이더 읽어와서 셰이더 프로그램 만들기

```
make_vertexShaders();
make_fragmentShaders();
shaderProgramID = make_shaderProgram();
```

//--- 렌더링 루프

```
while (!glfwWindowShouldClose(window))
{
    drawScene();
```

//--- 윈도우에 렌더링 결과 출력

```
glfwSwapBuffers(window);
```

//--- 이벤트 처리

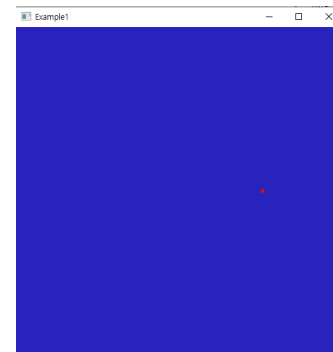
```
glfwPollEvents();
}
```

//--- 종료

```
glfwDestroyWindow(window);
glfwTerminate();
```

```
return 0;
```

```
}
```



셰이더 객체만들기

//--- 버텍스 셰이더 객체 만들기

```
void make_vertexShaders()
{
    //--- 셰이더 코드 읽어오기
    std::string vertexSource = filetobuf( " vertex.glsl");
    const char* source = vertexSource.c_str();

    //--- 셰이더 생성하기
    vertexShader = glCreateShader(GL_VERTEX_SHADER);

    //--- 셰이더에 코드 연결하고 컴파일 하기
    glShaderSource(vertexShader, 1, &source, NULL);
    glCompileShader(vertexShader);

    GLint result;
    GLchar errorLog[512];

    //--- 에러 체크하기
    glGetShaderiv (vertexShader, GL_COMPILE_STATUS, &result);
    if (!result)
    {
        glGetShaderInfoLog(vertexShader, 512, NULL, errorLog);
        std::cerr << "ERROR: vertex shader 컴파일 실패\n" << errorLog << std::endl;
        return;
    }
}
```

//--- 프래그먼트 셰이더 객체 만들기

```
void make_fragmentShaders()
{
    //--- 셰이더 코드 읽어오기
    std::string fragmentSource = filetobuf("fragment.glsl");
    const char* source = fragmentSource.c_str();

    //--- 셰이더 생성하기
    fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);

    //--- 셰이더에 코드 연결하고 컴파일 하기
    glShaderSource(fragmentShader, 1, &source, NULL);
    glCompileShader(fragmentShader);

    GLint result;
    GLchar errorLog[512];

    //--- 에러 체크하기
    glGetShaderiv(fragmentShader, GL_COMPILE_STATUS, &result);

    if (!result)
    {
        glGetShaderInfoLog(fragmentShader, 512, NULL, errorLog);
        std::cerr << "ERROR: fragment shader 컴파일 실패\n" << errorLog << std::endl;
        return;
    }
}
```

셰이더 프로그램 만들기

//--- 셰이더 프로그램 만들고 셰이더 객체 링크하기

```
GLuint make_shaderProgram()
{
    GLint result;
    GLchar errorLog[512];

    //--- 셰이더 프로그램 생성
    GLuint shaderID = glCreateProgram();

    //--- 버텍스 셰이더와 프래그먼트 셰이더 연결
    glAttachShader(shaderID, vertexShader);
    glAttachShader(shaderID, fragmentShader);

    //--- 셰이더 프로그램 링크
    glLinkProgram(shaderID);

    //--- 링크가 끝났으므로 셰이더 객체 삭제
    glDeleteShader(vertexShader);
    glDeleteShader(fragmentShader);

    //--- 링크 성공 여부 확인
    glGetProgramiv(shaderID, GL_LINK_STATUS, &result);

    if (!result) {
        glGetProgramInfoLog(shaderID, 512, NULL, errorLog);

        std::cerr << "ERROR: shader program 연결 실패\n" << errorLog << std::endl;
        return 0;
    }

    //--- 셰이더 프로그램 사용
    glUseProgram(shaderID);

    return shaderID;
}
```

메인/출력 함수 만들기

//--- 출력 함수

```
void drawScene()
```

```
{
```

```
    GLfloat rColor, gColor, bColor;
```

```
    rColor = gColor = 0.0f;
```

```
    bColor = 1.0f;
```

//--- 배경색을 파란색으로 설정

```
    glClearColor(rColor, gColor, bColor, 1.0f);
```

```
    glClear(GL_COLOR_BUFFER_BIT);
```

//--- 셰이더 프로그램 사용

```
    glUseProgram(shaderProgramID);
```

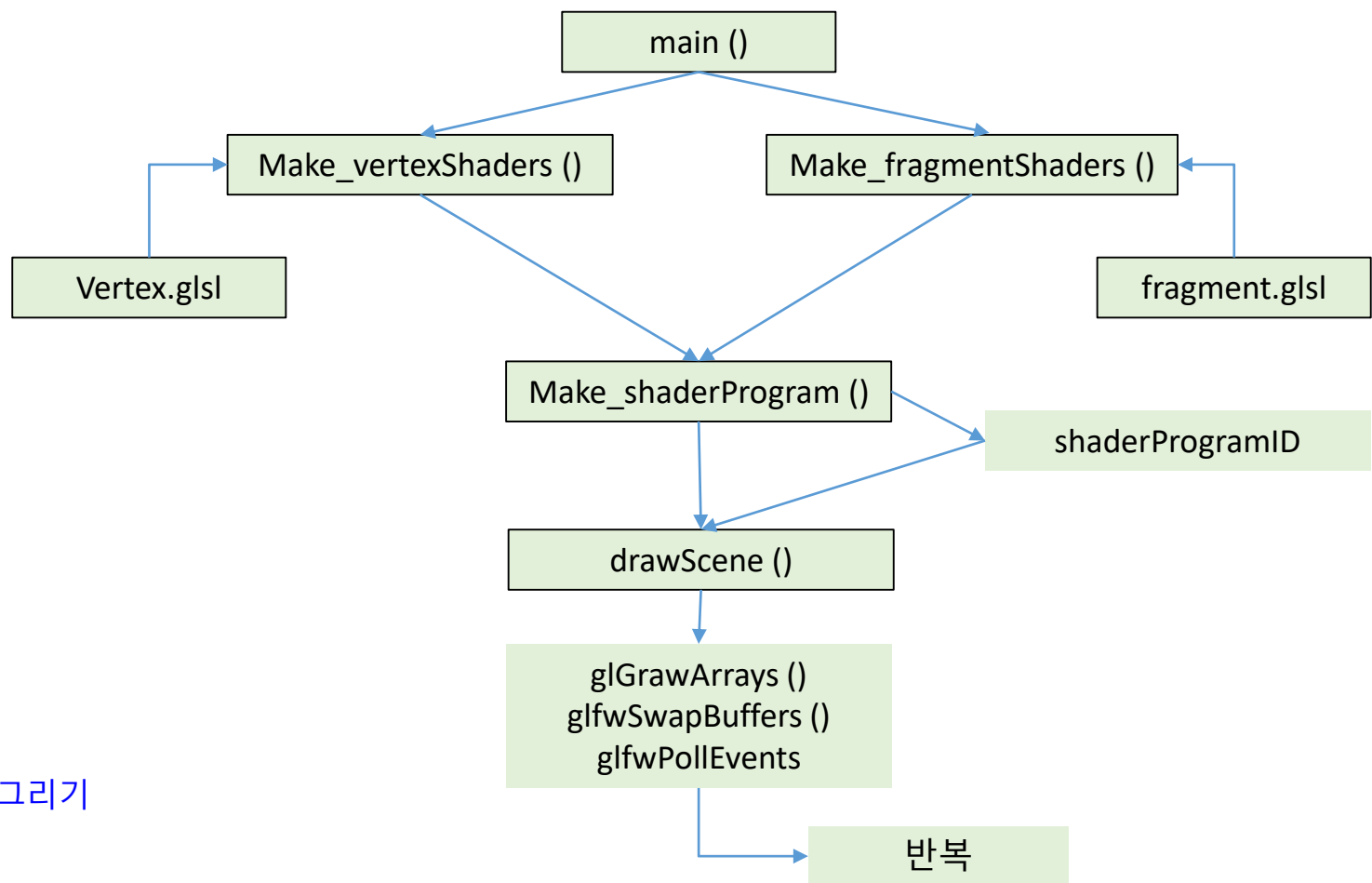
//--- 점의 크기 설정

```
    glPointSize(5.0f);
```

//--- 0번 인덱스에서 1개의 버텍스를 사용하여 점 그리기

```
    glDrawArrays(GL_POINTS, 0, 1);
```

```
}
```



파일에서 문자열 읽어오기 샘플

- 파일에서 문자열 읽어오기 샘플 코드 (워밍업에서 구현한 본인의 코드 사용 가능)

```
#include <fstream>
#include <string>

std::string filetoBuf(const char* file)
{
    std::ifstream shaderFile(file);

    if (!shaderFile.is_open())
    {
        return "";
    }

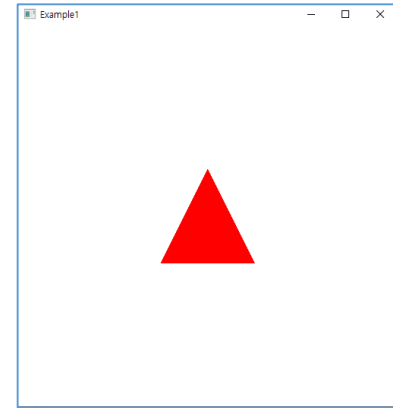
    //--- 파일 전체를 문자열로 읽기
    std::string source ( (std::istreambuf_iterator<char>(shaderFile)), std::istreambuf_iterator<char>());

    shaderFile.close();

    return source;
}
```

2. 두번째 셰이더 프로그램: 셰이더 사용하여 삼각형 그리기 (1)

- 중앙에 빨간색 삼각형 그리기:
 - 버텍스 셰이더: 셰이더 내에서 삼각형의 좌표값을 직접 설정
 - 프래그먼트 셰이더: 빨간색 고정하여 사용하기
- 화면 중앙에 삼각형 그리기
 - 버텍스 셰이더: 삼각형 좌표값 설정
 - 프래그먼트 셰이더: 도형 색상 설정
- 버텍스 셰이더와 프래그먼트 셰이더는 각각 별개의 파일로 저장하여 사용
 - 버텍스 셰이더: vertex.glsl
 - 프래그먼트 셰이더: fragment.glsl
- 메인 프로그램 작성하기
 - 화면 띄우기
 - 버텍스 셰이더 객체와 프래그먼트 셰이더 객체 만들기
 - 셰이더 프로그램 만들어 두 셰이더 객체 연결하기
 - 출력 콜백 함수에서 그리기 함수 호출하여 삼각형 그리기



버텍스 셰이더 코드: 삼각형 그리기

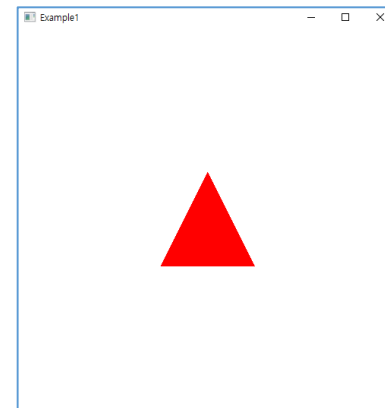
- 버텍스 셰이더: *vertex.glsl* 파일로 저장

```
#version 330 core
```

```
void main()
{
    const vec4 vertex[3] = vec4[3] (vec4(-0.25, -0.25, 0.5, 1.0),
                                     vec4(0.25, -0.25, 0.5, 1.0),
                                     vec4(0.0, 0.25, 0.5, 1.0));

    // const vec4 vertex[4] = vec4[4] (vec4(-0.25, -0.25, 0.5, 1.0),
    //                                  vec4(0.25, -0.25, 0.5, 1.0),
    //                                  vec4(0.0, 0.25, 0.5, 1.0),
    //                                  vec4(0.0, -0.5, 0.5, 1.0));

    gl_Position = vertex [gl_VertexID];
}
```



- **gl_VertexID**: 내장 변수로 해당 시점에 처리될 버텍스의 인덱스
 - gl_VertexID 입력값은 glDrawArrays()에 입력으로 들어간 첫 번째 인자값의 프리미티브 형태에 따라 두번째 인자값부터 시작해서 세 번째 인자인 count만큼의 버텍스까지 한 번에 하나씩 증가한다.
 - gl_VertexID의 값에 기반하여 각 버텍스에 다른 위치를 할당할 수 있다.
- **gl_Position**: 내장 변수로 해당 시점에 그릴 버텍스의 출력위치
vec4 (0.0, 0.0, 0.5, 1.0)을 할당하면 화면의 중앙에 위치한다.

프래그먼트 셰이더 코드: 삼각형 그리기

- 프래그먼트 셰이더: *fragment.glsl* 파일로 저장

```
#version 330 core
```

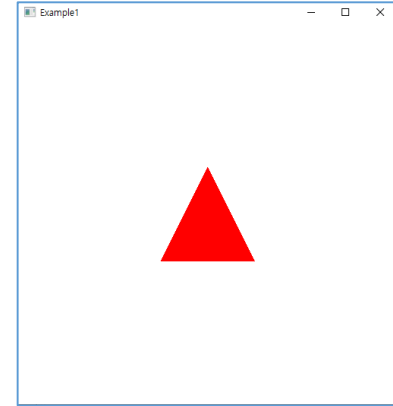
```
out vec4 color;
```

```
void main ()
```

```
{
```

```
    color = vec4 (1.0, 0.0, 0.0, 1.0);
```

```
}
```



셰이더 사용하여 삼각형 그리기

//--- 필요한 헤더파일 선언

```
#include <GL/glew.h>
#include <GL/glfw3.h>
```

//--- 사용자 정의 함수

```
void make_vertexShaders();
void make_fragmentShaders();
GLuint make_shaderProgram();
GLvoid drawScene();
GLvoid Reshape(int w, int h);
```

//--- 필요한 변수 선언

```
GLint width, height;
GLuint shaderProgramID;
GLuint vertexShader;
GLuint fragmentShader;
```

//--- 메인 함수

```
int main(int argc, char** argv)
{
    width = 500;
    height = 500;
```

//--- GLFW 초기화

```
if (!glfwInit())
    return -1;
```

//--- OpenGL 버전 및 프로파일 설정

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

//--- 윈도우 생성

```
GLFWwindow* window =
    glfwCreateWindow(width, height, "Example1", nullptr, nullptr);
```

if (!window)

```
{
    glfwTerminate();
    return -1;
}
```

//--- 생성한 윈도우의 OpenGL Context를 현재 Context로 설정

```
glfwMakeContextCurrent(window);
```

//--- GLEW 초기화

```
glewExperimental = GL_TRUE;
if (glewInit() != GLEW_OK)
{
    glfwDestroyWindow(window);
    glfwTerminate();
    return -1;
}
```

//--- 셰이더 읽어와서 셰이더 프로그램 만들기

```
make_vertexShaders();
make_fragmentShaders();
shaderProgramID = make_shaderProgram();
```

//--- 렌더링 루프

```
while (!glfwWindowShouldClose(window))
{
    drawScene();
```

//--- 윈도우에 렌더링 결과 출력

```
glfwSwapBuffers(window);
```

//--- 이벤트 처리

```
glfwPollEvents();
}
```

//--- 종료

```
glfwDestroyWindow(window);
glfwTerminate();
```

return 0;

```
}
```

셰이더 사용하여 삼각형 그리기

//--- 버텍스 셰이더 객체 만들기

```
void make_vertexShaders()
{
    std::string vertexSource = filetobuf("vertex.glsl");
    const char* source = vertexSource.c_str();

    vertexShader = glCreateShader(GL_VERTEX_SHADER);

    glShaderSource(vertexShader, 1, &source, NULL);
    glCompileShader(vertexShader);

    GLint result;
    GLchar errorLog[512];

    glGetShaderiv(vertexShader, GL_COMPILE_STATUS, &result);

    if (!result)
    {
        glGetShaderInfoLog(vertexShader, 512, NULL, errorLog);
        std::cerr << "ERROR: vertex shader 컴파일 실패\n" << errorLog << std::endl;

        return;
    }
}
```

//--- 프래그먼트 셰이더 객체 만들기

```
void make_fragmentShaders()
{
    std::string fragmentSource = filetobuf("fragment.glsl");
    const char* source = fragmentSource.c_str();

    fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);

    glShaderSource(fragmentShader, 1, &source, NULL);
    glCompileShader(fragmentShader);

    GLint result;
    GLchar errorLog[512];

    glGetShaderiv(fragmentShader, GL_COMPILE_STATUS, &result);

    if (!result)
    {
        glGetShaderInfoLog(fragmentShader, 512, NULL, errorLog);

        std::cerr << "ERROR: fragment shader 컴파일 실패\n" << errorLog << std::endl;

        return;
    }
}
```

셰이더 사용하여 삼각형 그리기

//--- 셰이더 프로그램 만들고 셰이더 객체 링크하기

```
GLuint make_shaderProgram()
{
    GLint result;
    GLchar errorLog[512];

    //--- 셰이더 프로그램 생성, 연결, 링크, 셰이더 객체 삭제
    GLuint shaderID = glCreateProgram();

    glAttachShader(shaderID, vertexShader);
    glAttachShader(shaderID, fragmentShader);

    glLinkProgram(shaderID);

    glDeleteShader(vertexShader);
    glDeleteShader(fragmentShader);

    //--- 링크 성공 여부 확인
    glGetProgramiv(shaderID, GL_LINK_STATUS, &result);

    if (!result) {
        glGetProgramInfoLog(shaderID, 512, NULL, errorLog);

        std::cerr << "ERROR: shader program 연결 실패\n" << errorLog << std::endl;
        return 0;
    }

    //--- 셰이더 프로그램 사용
    glUseProgram(shaderID);

    return shaderID;
}
```

셰이더 사용하여 삼각형 그리기

//--- 출력 함수

void drawScene()

{

GLfloat rColor, gColor, bColor;

rColor = gColor = 0.0f;

bColor = 1.0f;

//--- 배경색을 파란색으로 설정

glClearColor(rColor, gColor, bColor, 1.0f);

glClear(GL_COLOR_BUFFER_BIT);

//--- 셰이더 프로그램 사용

glUseProgram(shaderProgramID);

//--- 점의 크기 설정

glPointSize(5.0f);

//--- 0번 인덱스에서 3개의 버텍스를 사용하여 삼각형 그리기

glDrawArrays(GL_POINTS, 0, 3);

// glDrawArrays(GL_TRIANGLES, 0, 3);

// glDrawArrays(GL_LINE_LOOP, 0, 3);

}

3. 세번째 셰이더 프로그램: 셰이더 사용하여 삼각형 그리기 (2)

- 응용 프로그램에서 버텍스 데이터를 셰이더로 보내서 삼각형 그리기

- 이전 예제에서는, 좌표값과 색상이 셰이더 코드에 고정 → 실제로는 많은 정점 및 정보가 있을 수 있다.
- 따라서, 위치, 색상 등의 속성을 셰이더에서 고정하지 않고, GPU 메모리에 따로 저장

- **Vertex Buffer Objects (VBO)**

- 버텍스 데이터를 GPU 메모리에 저장해 놓는 버퍼 (배열을 사용)
- GL buffer object는 이 버텍스 배열을 저장, 초기화, 렌더링을 할 수 있게 한다.

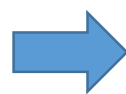
- **Vertex Array Object (VAO)**

- Vertex Attribute 설정을 저장하는 객체
 - 하나의 오브젝트를 구성하는 속성들을 개별 VBO에 저장하고 VAO로 묶어서 사용한다.
 - VAO에는 버텍스 데이터가 직접 저장되는 것이 아니라 연결 정보만 저장
 - VAO는 VBO에 저장된 데이터 타입과 어떤 속성 변수가 데이터를 가져가게 되는지 저장

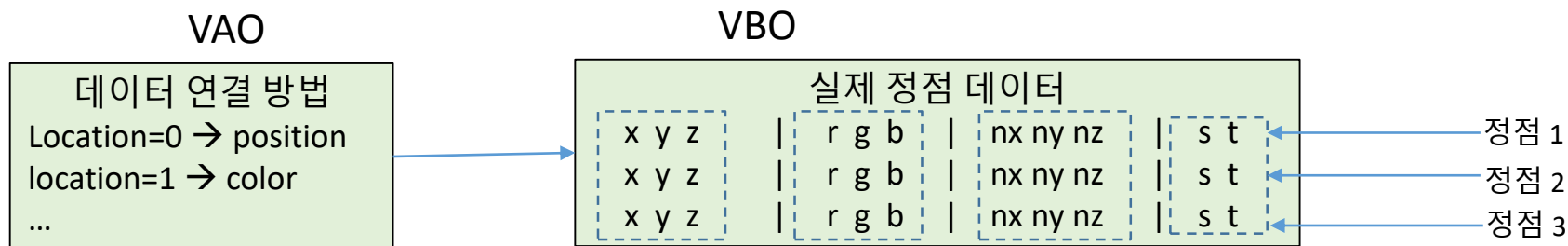
- VBO: 데이터 저장
- VAO: 데이터 연결 방법 저장

Vertex Specifications

- 버텍스 속성 (Vertex Attributes)
 - 정점 하나가 가지고 있는 데이터의 종류, 즉 정점의 속성
 - 예를 들어, 위치 (position), 색상 (color), 법선벡터 (normal), 텍스처 좌표 (texture coordinate) 속성
- 이러한 데이터들은 VBO에 저장된다.
- 각 Vertex Attribute에는 location 번호를 부여한다 → layout 한정자 사용
- 예를 들어,
 - location 0 → Position
 - location 1 → Color
 - location 2 → Normal
 - location 3 → TexCoord

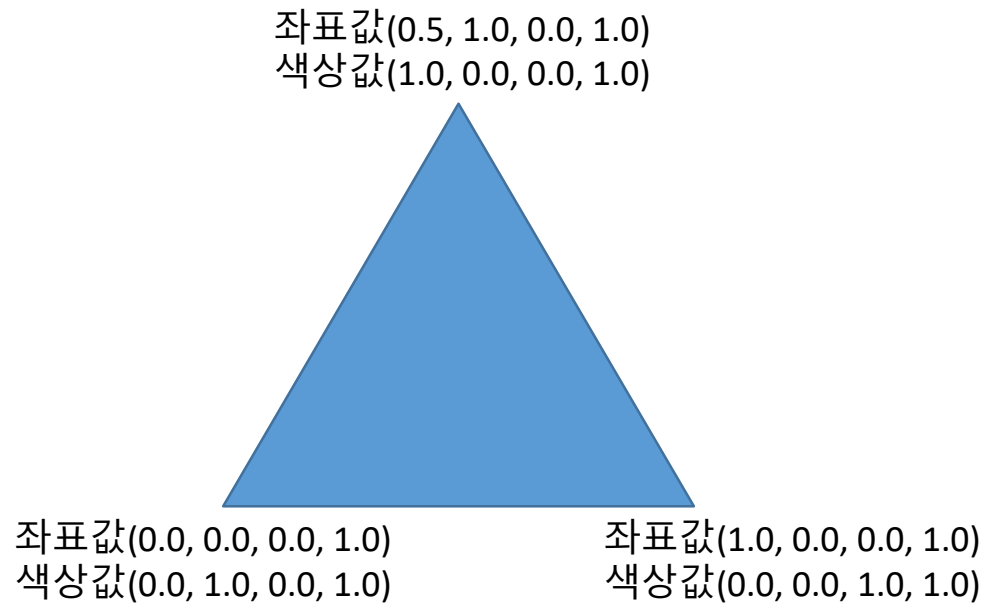


layout(location = 0) in vec3 vPosition; (Attribute 0 → vPosition)
layout(location = 1) in vec3 vColor; (Attribute 1 → vColor)



Vertex Specifications

- 버텍스의 속성들
 - 위치 (position)
 - 색상 (color)
 - 텍스처 좌표 (texture coordinates)
 - 노말 (normal)
 - 그 외 데이터 들



```
//--- vertex attribute: position
```

```
const float vertexPosition [] =  
{  
    0.5, 1.0, 0.0,  
    0.0, 0.0, 0.0,  
    1.0, 0.0, 0.0  
};
```

```
//--- vertex attribute: color
```

```
const float vertexColor [] =  
{  
    1.0, 0.0, 0.0,  
    0.0, 1.0, 0.0,  
    0.0, 0.0, 1.0  
};
```

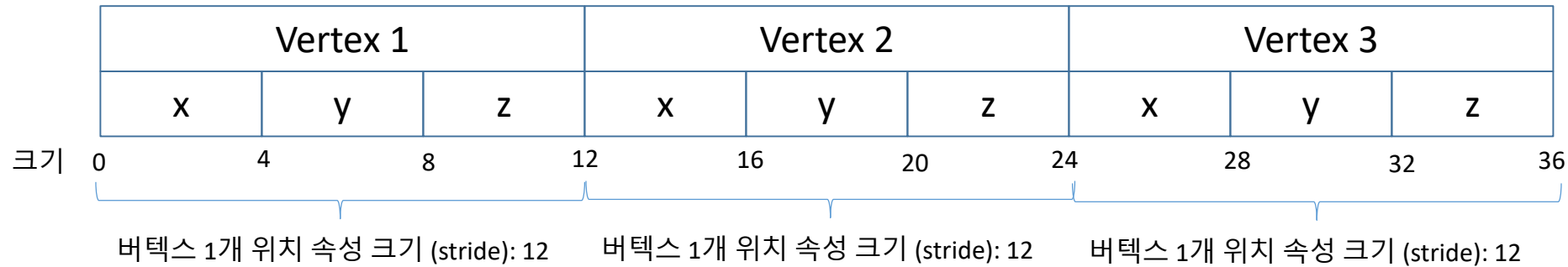
```
// 두 속성을 한 개의 배열에 저장 -----
```

```
//--- vertex attributes: position color
```

```
const float vertexData [] =  
{  
    0.5, 1.0, 0.0,    1.0, 0.0, 0.0,  
    0.0, 0.0, 0.0,    0.0, 1.0, 0.0,  
    1.0, 0.0, 0.0,    0.0, 0.0, 1.0  
};
```

Vertex Attribute

- 버텍스 속성 연결
 - 입력 데이터의 어느 부분이 vertex shader의 어떠한 정점 속성과 맞는지 직접 지정해야 함
- 예) 삼각형의 위치 값
 - 한 개의 버텍스는 3개의 실수 값 (x, y, z 좌표값)으로 구성
 - 각 실수값은 4바이트로 구성
 - 즉, 한 개의 버텍스 속성 값(위치)은 12 바이트 (3개 실수 * 4바이트)로 구성



- 위의 속성 값을 설정해야 함

Vertex Buffer Object

- **Vertex Buffer Object (VBO)**

- 버텍스 데이터를 저장하기 위한 버퍼 객체
 - 버텍스의 다양한 속성들을 저장한다.
 - Position, Normal, Normal, Color 등
 - 하나의 VBO에 여러 종류의 속성 (attribute)를 저장할 수 있음
- 대용량 버텍스 데이터를 GPU 메모리에 저장하여 효율적으로 처리할 수 있음

- 바인드(bind)란
오픈지엘에서 앞으로 작업할 객체를 현재 대상으로 지정하는 것

- 버퍼를 생성하고 바인드하고 실제 데이터를 넣어줌

```
GLuint VBO;
```

```
glGenBuffers (1, &VBO);
```

```
glBindBuffer (GL_ARRAY_BUFFER, VBO);
```

```
glBufferData (GL_ARRAY_BUFFER, sizeof (vertexPosition), vertexPosition, GL_STATIC_DRAW);
```

```
//--- 버퍼 id를 생성
```

```
//--- 버퍼 객체에 저장할 데이터 타입 지정
```

```
//--- 바인드 후에 호출하는 모든 버퍼는 바인딩 된 버퍼를 사용한다.
```

```
//--- 사용자가 정의한 데이터를 현재 바인딩된 버퍼에 복사한다.
```

- 현재 바인딩된 버퍼가 몇 번째 attribute 버퍼인지 어떤 속성을 갖는지 알려주기위해 속성 포인터 설정하고 사용

```
glVertexAttribPointer (0, 3, GL_FLOAT, GL_FALSE, 0, 0);
```

```
glEnableVertexAttribArray (0);
```

Vertex Array Object

• Vertex Array Object (VAO)

- Vertex Attribute와 VBO의 연결 및 Vertex Attribute 설정 정보를 저장하는 객체
- 하나 또는 여러 개의 VBO에 저장된 Vertex Attribute을 하나의 VAO로 관리할 수 있음
- VAO에는 버텍스 데이터 자체가 저장되는 것이 아니라 연결 및 설정 정보가 저장됨
 - VBO에 저장된 데이터의 형식, 위치(location), 데이터 읽기 방법 등을 저장
- 보통 하나의 Mesh마다 하나의 VAO를 사용하는 방식이 많이 사용됨

• VAO를 생성하고 바인드 함

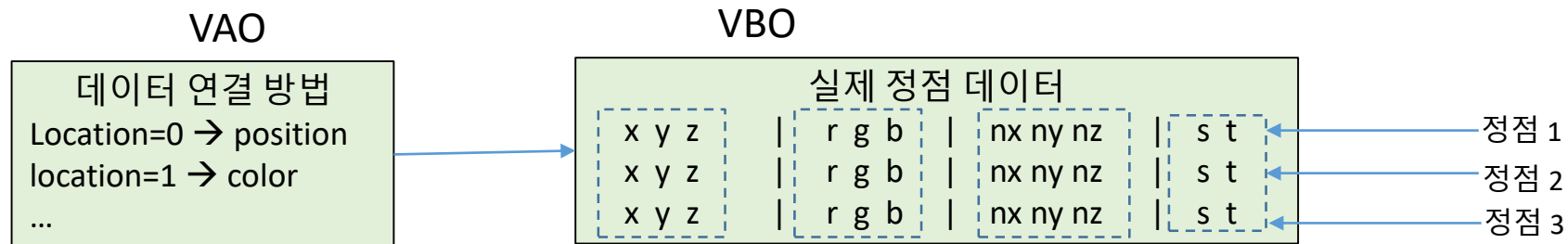
GLuint VAO;

glGenVertexArrays (1, & VAO);

glBindVertexArray (VAO);

//--- 버텍스 array 생성

//--- VAO를 가진다는 의미이고 아직 실제 데이터는 없음



함수 프로토타입

- Vertex Array Object (VAO), Vertex Buffer Object (VBO) 생성 및 바인딩 함수
 - void **glGenBuffers** (GLsizei n, GLuint *buffers);
 - 버퍼 오브젝트 (VBO) 이름 생성
 - n: 생성할 이름 개수
 - buffers: 버퍼 오브젝트 이름을 가리키는 배열
 - void **glBindBuffer** (GLenum target, GLuint buffer);
 - 버퍼 오브젝트를 바인드 한다.
 - Target: 바인드할 버퍼 타겟 타입
 - GL_ARRAY_BUFFER: 버텍스 속성
 - GL_ELEMENT_ARRAY_BUFFER: 버텍스 배열 인덱스
 - GL_TEXTURE_BUFFER: 텍스처 데이터 버퍼
 - Buffer: 버퍼 오브젝트 이름
 - void **glBufferData** (GLenum target, GLsizeiptr size, const GLvoid *data, GLenum usage);
 - 버퍼 오브젝트의 데이터를 생성
 - target: 바인드할 버퍼 타겟 타입
 - GL_ARRAY_BUFFER, GL_ELEMENT_ARRAY_BUFFER, GL_TEXTURE_BUFFER...
 - size: 버퍼 오브젝트의 크기
 - data: 저장할 데이터를 가리키는 포인터
 - usage: 저장한 데이터를 사용할 패턴
 - GL_STATIC_DRAW: 한번 버텍스 데이터 업데이트 후 변경이 없는 경우 사용
 - GL_STREAM_DRAW: 데이터가 그려질 때마다 변경
 - GL_DYNAMIC_DRAW: 버텍스 데이터가 자주 바뀌는 경우 (애니메이션), 버텍스 데이터가 바뀔 때마다 다시 업로드 된다

함수 프로토타입

- void **glGenVertexArrays** (GLsizei n, GLuint *arrays);
 - 버텍스 배열 오브젝트 (VAO) 이름 생성
 - n: 생성할 VAO 개수
 - arrays: VAO 저장할 배열 이름
- void **glBindVertexArray** (GLuint array);
 - VAO를 바인드한다.
 - array: 바인드할 버텍스 배열의 이름
- void **glBufferSubData** (GLenum target, GLintptr offset, GLsizeiptr size, const void *data);
 - 이미 생성되어 있는 버퍼 객체의 데이터 중 일부를 새로운 데이터로 교체
 - target: 바인드할 버퍼 타겟 타입
 - GL_ARRAY_BUFFER, GL_ELEMENT_ARRAY_BUFFER, GL_TEXTURE_BUFFER...
 - offset: 데이터를 변경하기 시작할 위치 (바이트 단위)
 - size: 변경할 데이터의 크기 (바이트 단위)
 - data: CPU 메모리에 있는 새로운 데이터의 주소
 - 사용 예)

```
glBindBuffer (GL_ARRAY_BUFFER, vbo[0]);  
glBufferSubData(GL_ARRAY_BUFFER, 0, sizeof(triShape), triShape);
```

함수 프로토타입

- 정점 속성 설정 함수

- void **glVertexAttribPointer** (GLuint index, GLint size, GLenum type, GLboolean normalized, GLsizei stride, const GLvoid *pointer)
 - 버텍스 속성 데이터의 배열을 정의
 - index: 설정할 vertex 속성의 인덱스 값을 지정. (셰이더에서 layout (location = 0) → 속성의 위치가 0번째)
 - size: 버텍스 속성의 크기 (버텍스 속성이 vec3라면 3)
 - type: 데이터 타입 (vec3 라면 GL_FLOAT)
 - normalized: 데이터를 정규화할지 (GL_TRUE: [0, 1] 사이의 값으로 정규화, GL_FALSE: 그대로 사용)
 - stride: 연이은 vertex 속성 세트들 사이의 공백 (값이 공백없이 채워져 있다면 0, 1개 이상의 속성들이 저장되어 있다면 크기를 설정. 예) 버텍스 vec3라면 다음 버텍스 위치는 12바이트)
 - pointer: 데이터가 시작하는 위치의 오프셋 값
 - 각 vertex 속성은 VBO에 의해 관리되는 메모리로부터 데이터를 받는다.
 - 데이터를 받을 VBO (하나가 여러 VBO를 가질 수도 있음)는 glVertexAttribPointer 함수를 호출할 때 GL_ARRAY_BUFFER에 현재 바인딩된 VBO로 결정
 - 사용 예) glVertexAttribPointer (0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
- void **glEnableVertexAttribArray** (GLuint index);
 - 버텍스 속성 배열을 사용하도록 한다.
 - index: 버텍스 속성 인덱스
 - 사용 예) glEnableVertexAttribArray (0);

1) Vertex Attribute 1개 인 경우

- 한 개의 VBO와 한 개의 VAO 사용하기 예)

- Vertex shader

```
#version 330 core
```

```
layout (location = 0) in vec3 vPos;           //--- 1개의 속성 - attribute로 설정된 위치 속성: 인덱스 0
```

```
void main()
```

```
{
```

```
    gl_Position = vec4 (vPos.x, vPos.y, vPos.z, 1.0);
```

```
}
```

- Fragment shader

```
#version 330 core
```

```
out vec4 FragColor;                          //--- 출력할 객체의 색상
```

```
void main()
```

```
{
```

```
    FragColor = vec4 (1.0f, 0.5f, 0.3f, 1.0f);
```

```
}
```

1) Vertex Attribute 1개 인 경우

- 버텍스 위치 속성 설정하기 응용 프로그램 예)

//--- 변수 선언

GLuint VAO, VBO_position;

GLvoid InitBuffer ()

{

//--- VAO와 VBO 객체 생성

glGenVertexArrays (1, &VAO);

glGenBuffers (1, &VBO_position);

//--- 사용할 VAO 바인딩

glBindVertexArray (VAO);

//--- vertex positions 저장할 위한 VBO 바인딩

glBindBuffer (GL_ARRAY_BUFFER, VBO_position);

//--- vertex positions 데이터 저장

glBufferData (GL_ARRAY_BUFFER, sizeof (vertexPosition), vertexPosition, GL_STATIC_DRAW);

//--- 현재 바인딩되어있는 VBO를 0번째 attribute에 가져오도록 지정하고 그 인덱스의 attribute를 활성화

glVertexAttribPointer (0, 3, GL_FLOAT, GL_FALSE, 3*sizeof(float), 0);

glEnableVertexAttribArray (0);

}

```
const float vertexPosition [] =  
{  
    0.5, 1.0, 0.0,  
    0.0, 0.0, 0.0,  
    1.0, 0.0, 0.0  
};
```

```
const float vertexColor [] =  
{  
    1.0, 0.0, 0.0,  
    0.0, 1.0, 0.0,  
    0.0, 0.0, 1.0  
};
```

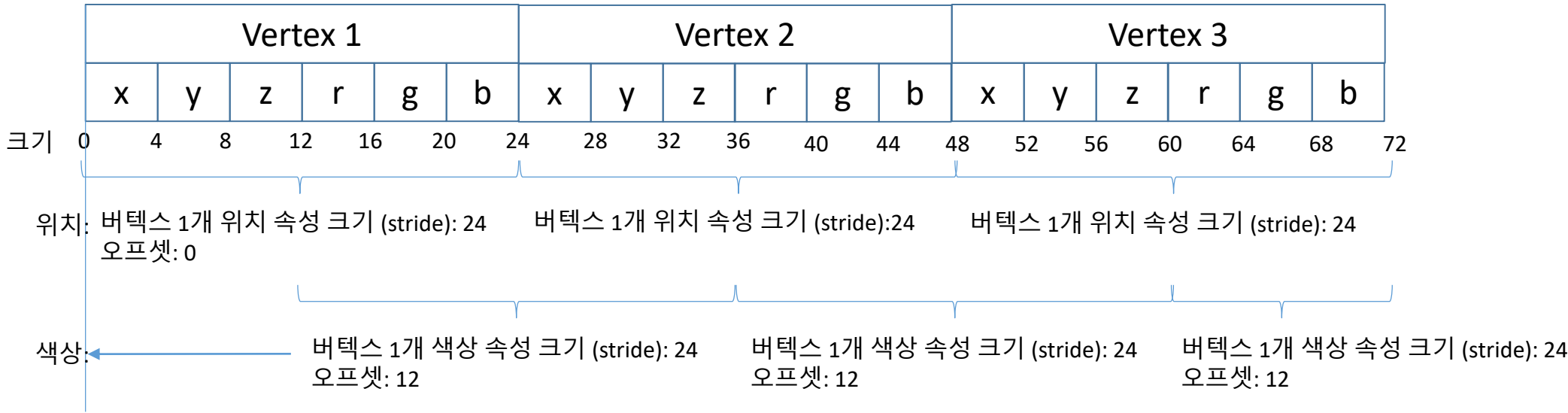
// 위치와 색상을 한 개의 데이터로 저장

```
const float vertexData [] =  
{  
    0.5, 1.0, 0.0,      1.0, 0.0, 0.0,  
    0.0, 0.0, 0.0,      0.0, 1.0, 0.0,  
    1.0, 0.0, 0.0,      0.0, 0.0, 1.0  
};
```

2) Vertex Attribute 2개인 경우

• 버텍스 포맷에 속성 추가

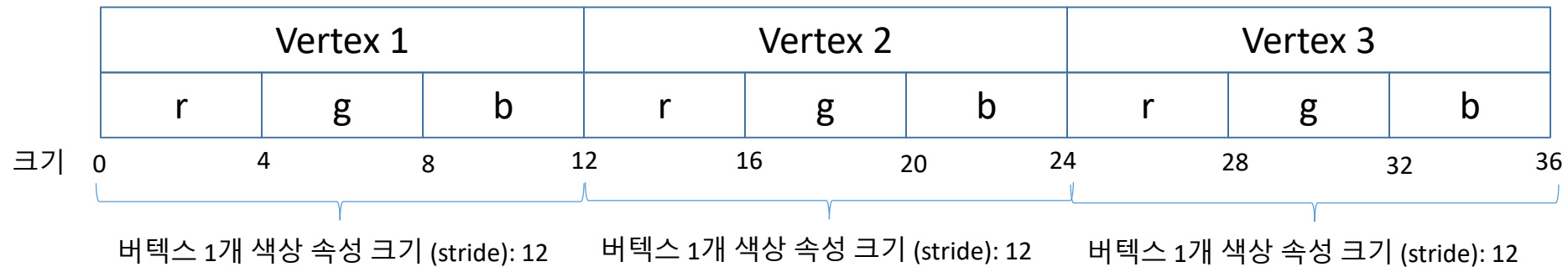
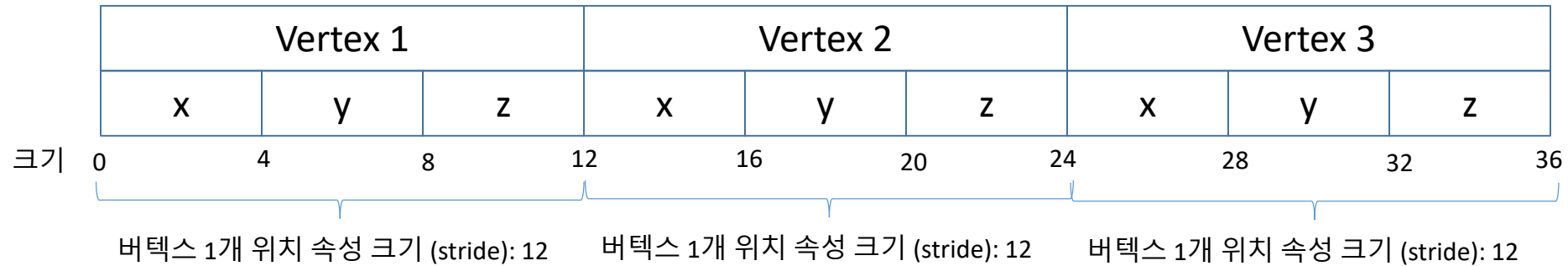
① 좌표값 외에 색상 값을 속성으로 추가하는 경우: 두 개의 속성을 같이 저장하는 경우



2) Vertex Attribute 2개인 경우

- 버텍스 포맷에 속성 추가

② 좌표값 외에 색상값을 속성으로 추가하는 경우: 두 개의 속성을 따로 저장하는 경우



2-1) Vertex Attribute: 2개 속성을 한 변수로 저장 (앞의 1번 그림)

- 속성 추가하여 1개 이상의 속성을 사용 하는 경우 (VertexData 변수 사용)

(54페이지)

- Vertex Shader

```
#version 330 core
layout (location = 0) in vec3 vPos;      //--- 위치 변수: attribute position 0
layout (location = 1) in vec3 vColor;    //--- 컬러 변수: attribute position 1

out vec3 outColor;                       //--- 컬러를 fragment shader로 출력

void main()
{
    gl_Position = vec4 (vPos, 1.0);
    outColor = vColor;                   //--- vertex data로부터 가져온 컬러 입력을 outColor에 설정
}
```

- Fragment Shader

```
#version 330 core
out vec4 FragColor;
in vec3 outColor;

void main()
{
    FragColor = vec4 (outColor, 1.0);
}
```

2-1) Vertex Attribute: 2개 속성을 한 변수로 저장

- 응용 프로그램

//--- 변수 선언

GLuint VAO, VBO;

void InitBuffer ()

{

//--- VAO 객체 생성 및 바인딩

glGenVertexArrays (1, &VAO);

glBindVertexArray (VAO);

//--- vertex data 저장을 위한 VBO 생성 및 바인딩.

glGenBuffers (1, &VBO);

glBindBuffer (GL_ARRAY_BUFFER, VBO);

//--- vertex data 데이터 입력.

glBufferData (GL_ARRAY_BUFFER, sizeof (vertexData), vertexData, GL_STATIC_DRAW);

//--- 위치 속성: 속성 위치 0

glVertexAttribPointer (0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);

glEnableVertexAttribArray (0);

//--- 색상 속성: 속성 위치 1

glVertexAttribPointer (1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 * sizeof(float)));

glEnableVertexAttribArray (1);

}

```
const float vertexPosition [] =
{
    0.5, 1.0, 0.0,
    0.0, 0.0, 0.0,
    1.0, 0.0, 0.0
};
const float vertexColor [] =
{
    1.0, 0.0, 0.0,
    0.0, 1.0, 0.0,
    0.0, 0.0, 1.0
};
```

// 위치와 색상을 한 개의 데이터로 저장

```
const float vertexData [] =
{
    0.5, 1.0, 0.0,      1.0, 0.0, 0.0,
    0.0, 0.0, 0.0,      0.0, 1.0, 0.0,
    1.0, 0.0, 0.0,      0.0, 0.0, 1.0
};
```

void glVertexAttribPointer (GLuint index, GLint size, GLenum type, GLboolean normalized, GLsizei stride, const GLvoid *pointer)

- Index: 설정할 vertex 속성의 인덱스
- Size: 버텍스 속성의 크기
- Type: 데이터 타입
- Normalized: 데이터를
- Stride: 연이은 vertex 속성 세트들 사이의 공백 (값이 공백없이 채워져 있다면 0, 1개 이상의 속성들이 저장되어 있다면 크기를 설정.
- Pointer: 데이터가 시작하는 위치의 오프셋 값

2-2) Vertex Attribute: 2개 속성을 따로 저장할 때 (앞의 2번 경우)

(55페이지)

- VBO를 2개 생성
 - 각 VBO에 속성값 (위치, 색상) 저장
 - 셰이더에 위치와 색상 저장하기
- VAO를 생성
 - VAO에 2개의 속성을 바인드하기

• Vertex Shader

```
#version 330 core
```

```
in vec3 vPos;           //--- 메인 프로그램에서 입력 받음
in vec3 vColor;         //--- 메인 프로그램에서 입력 받음
out vec3 passColor;     //--- fragment shader로 전달
```

```
void main()
{
    gl_Position = vec4 (vPos.x, vPos.y, vPos.z, 1.0);
    passColor = vColor;
}
```

• Fragment Shader

```
#version 330 core
```

```
in vec3 passColor;      //--- vertex shader에서 입력 받음
out vec4 FragColor;     //--- 프레임 버퍼로 출력
```

```
void main()
{
    FragColor = vec4 (passColor, 1.0);
}
```

2-2) Vertex Attribute: 2개 속성을 따로 저장할 때

- 응용 프로그램

//--- 변수 선언

GLuint VAO, VBO_position, VBO_color;

void InitBuffer ()

{

//--- Vertex Array Object 생성

glGenVertexArrays (1, &VAO);

glBindVertexArray (VAO);

//--- 위치 속성

glGenBuffers (1, &VBO_position);

glBindBuffer (GL_ARRAY_BUFFER, VBO_position);

glBufferData (GL_ARRAY_BUFFER, sizeof(vertexPosition), vertexPosition, GL_STATIC_DRAW);

//--- 색상 속성

glGenBuffers (1, &VBO_color);

glBindBuffer (GL_ARRAY_BUFFER, VBO_color);

glBufferData (GL_ARRAY_BUFFER, sizeof(vertexColor), vertexColor, GL_STATIC_DRAW);

//--- vPos 속성 변수에 값을 저장

GLint pAttribute = glGetAttribLocation (shaderProgramID, "vPos");

glBindBuffer (GL_ARRAY_BUFFER, VBO_position);

glVertexAttribPointer (pAttribute, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), 0);

glEnableVertexAttribArray (pAttribute);

//--- vColor 속성 변수에 값을 저장

GLint cAttribute = glGetAttribLocation (shaderProgramID, "vColor");

glBindBuffer (GL_ARRAY_BUFFER, VBO_color);

glVertexAttribPointer (cAttribute, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), 0);

glEnableVertexAttribArray (cAttribute);

}

```
const float vertexPosition [] =
```

```
{
```

```
    0.5, 1.0, 0.0,
```

```
    0.0, 0.0, 0.0,
```

```
    1.0, 0.0, 0.0
```

```
};
```

```
const float vertexColor [] =
```

```
{
```

```
    1.0, 0.0, 0.0,
```

```
    0.0, 1.0, 0.0,
```

```
    0.0, 0.0, 1.0
```

```
};
```

// 위치와 색상을 한 개의 데이터로 저장

```
const float vertexData [] =
```

```
{
```

```
    0.5, 1.0, 0.0,
```

```
    1.0, 0.0, 0.0,
```

```
    0.0, 0.0, 0.0,
```

```
    0.0, 1.0, 0.0,
```

```
    1.0, 0.0, 0.0,
```

```
    0.0, 0.0, 1.0
```

```
};
```

함수 프로토타입

- 위치 가져오기 함수
 - GLint **glGetAttribLocation** (GLuint program, const GLchar *name);
 - Attribute 변수의 위치를 가져온다.
 - 리턴값: 속성 변수의 위치
 - 위치를 찾지 못하면 음수값을 리턴한다.
 - Program: 프로그램 이름
 - Name: 위치를 찾으려는 attribute 변수 이름

3) 프래그먼트 셰이더로 직접 데이터 전달하기

- **Uniform** 한정자를 사용하여 원하는 셰이더로 직접 데이터 전달 가능
 - Uniform: CPU의 응용 프로그램에서 GPU의 셰이더로 데이터를 전달하는 방법
 - Uniform은 global 변수
 - 변수 앞에 uniform 키워드를 붙여 변수 선언
- 유니폼 변수 다루기
 - GLint **glGetUniformLocation** (GLuint program, const GLchar *name);
 - 프로그램에서 uniform 변수의 위치를 가져온다.
 - program: 셰이더 프로그램 이름
 - name: uniform 변수 이름
 - 리턴값: uniform 변수 위치 (-1: 위치를 찾지 못함)
 - void **glUniform{1|2|3|4}{f|i|ui}** (GLuint location, {GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3});
 - glUniform1f, glUniform2f, glUniform3f, glUniform4f...
 - 현재 프로그램에서 uniform 변수의 값을 명시
 - location: 수정할 uniform 변수의 위치
 - v0, v1, v2, v3: 사용될 uniform 변수 값

3) 프래그먼트 셰이더로 직접 데이터 전달하기

- 응용 프로그램

//--- 그리기 출력 콜백 함수에서 프래그먼트 셰이더로 컬러 값을 보내기

```
void drawScene ()
```

```
{
```

```
    ...
```

```
    //--- uniform 변수의 인덱스 값
```

```
    int vColorLocation = glGetUniformLocation (shaderProgramID, "vColor");
```

```
    //--- uniform 변수가 있는 프로그램 활성화:
```

```
    glUseProgram (shaderProgramID);
```

```
    //--- uniform 변수의 위치에 변수의 값 설정
```

```
    glUniform4f (vColorLocation, 1.0f, 0.0f, 0.0f, 1.0f);
```

```
    //--- 필요한 드로잉 함수 호출
```

```
    glBindVertexArray(VAO);
```

```
    glDrawArrays(GL_TRIANGLES, 0, 3);
```

```
}
```

3) 프래그먼트 셰이더로 직접 데이터 전달하기

- 프래그먼트 셰이더에 색상 직접 전달하기 예)

- Vertex shader

```
#version 330 core
```

```
layout (location = 0) in vec3 vPos;    ///--- attribute로 설정된 위치 속성: 인덱스 0
```

```
void main()
```

```
{
```

```
    gl_Position = vec4 (vPos.x, vPos.y, vPos.z, 1.0);
```

```
}
```

- Fragment shader

```
#version 330 core
```

```
uniform vec4 vColor;                ///--- 응용 프로그램에서 변수 값 설정
```

```
out vec4 FragColor;                  ///--- 출력할 객체의 색상
```

```
void main()
```

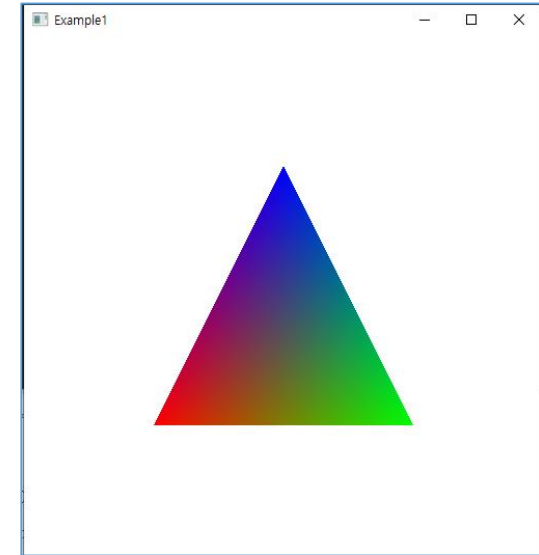
```
{
```

```
    FragColor = vColor;
```

```
}
```

최종 예) 위치, 색상 속성을 사용하여 화면 중앙에 삼각형 그리기

- Main (): 관련 콜백 함수 지정
 - 윈도우 띄우기
 - 필요한 콜백 함수 지정
 - 이벤트 루프 시작
- InitBuffer ():
 - VAO, VBO 만들기
 - 속성 (attributes) 설정하기
- InitShader ():
 - 셰이더 객체 만들기, 컴파일 하기, 셰이더 프로그램 만들고 링크하기
- 콜백 함수들
 - 각종 이벤트 처리하기
 - 출력 함수: display 콜백 함수
 - 화면에 출력하기 (glDrawArrays 또는 glDrawElements)
- 셰이더 만들기
 - Vertex shader, fragment shader 파일 작성



```
const GLfloat triShape[3][3] = {           // 삼각형 꼭지점 좌표값
    {-0.5, -0.5, 0.0},
    { 0.5, -0.5, 0.0 },
    { 0.0,  0.5, 0.0} };

const GLfloat colors[3][3] = {             // 삼각형 꼭지점 색상
    {1.0,  0.0,  0.0},
    {0.0,  1.0,  0.0},
    {0.0,  0.0,  1.0} };
```

최종 예) 위치, 색상 속성을 사용하여 화면 중앙에 삼각형 그리기

//--- vertex shader: vertex.glsl 파일에 저장

```
#version 330 core
//--- Position: attribute index 0
//--- Color: attribute index 1
```

```
layout (location = 0) in vec3 vPosition; //--- 위치 변수: attribute position 0
layout (location = 1) in vec3 vColor; //--- 컬러 변수: attribute position 1
out vec3 out_Color; //--- 프래그먼트 셰이더에게 전달
```

```
void main(void)
{
    gl_Position = vec4 (vPosition 1.0);

    out_Color = vColor;
}
```

//--- fragment shader: fragment.glsl 파일에 저장

```
#version 330 core
//--- out_Color: 버텍스 셰이더에서 전달받는 색상 값
//--- FragColor: 출력할 색상의 값으로 프레임 버퍼로 전달 됨.
```

```
in vec3 out_Color; //--- 버텍스 셰이더에게서 전달 받음
out vec4 FragColor; //--- 색상 출력
```

```
void main(void)
{
    FragColor = vec4 (out_Color, 1.0);
}
```

최종 예) 위치, 색상 속성을 사용하여 화면 중앙에 삼각형 그리기

//--- 필요한 헤더파일 선언

```
#include <GL/glew.h>
#include <GL/glfw3.h>
```

//--- 사용자 정의 함수

```
void make_vertexShaders();
void make_fragmentShaders();
int make_shaderProgram();
GLvoid drawScene();
GLvoid Reshape(int w, int h);
```

//--- 필요한 변수 선언

```
GLint width, height;
GLuint shaderProgramID;
GLuint vertexShader;
GLuint fragmentShader;
```

//--- 메인 함수

```
int main (int argc, char** argv)
{
    width = 500;
    height = 500;
```

//--- GLFW 초기화

```
if (!glfwInit())
    return -1;
```

//--- OpenGL 버전 및 프로파일 설정

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

//--- 윈도우 생성

```
GLFWwindow* window =
    glfwCreateWindow(width, height, "Example1", nullptr, nullptr);
```

if (!window)

```
{
    glfwTerminate();
    return -1;
}
```

//--- 생성한 윈도우의 OpenGL Context를 현재 Context로 설정
glfwMakeContextCurrent(window);

//--- GLEW 초기화

```
glewExperimental = GL_TRUE;
if (glewInit() != GLEW_OK)
{
    glfwDestroyWindow(window);
    glfwTerminate();
    return -1;
}
```

//--- 셰이더 읽어와서 셰이더 프로그램 만들기

```
make_vertexShaders();
make_fragmentShaders();
shaderProgramID = make_shaderProgram();
```

//--- VBO, VAO 생성하고 설정하기

```
InitBuffer ();
```

//--- 렌더링 루프

```
while (!glfwWindowShouldClose(window))
{
    drawScene();
```

//--- 윈도우에 렌더링 결과 출력

```
glfwSwapBuffers(window);
```

//--- 이벤트 처리

```
glfwPollEvents();
}
```

//--- 종료

```
glfwDestroyWindow(window);
glfwTerminate();
```

```
return 0;
```

```
}
```

최종 예) 위치, 색상 속성을 사용하여 화면 중앙에 삼각형 그리기

//--- 그리기 콜백 함수

```
GLvoid drawScene()
{
    //--- 변경된 배경색 설정
    glClearColor(rColor, gColor, bColor, 1.0f);
    //glClearColor(1.0, 1.0, 1.0, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    //--- 렌더링 파이프라인에 셰이더 불러오기
    glUseProgram(shaderProgramID);

    //--- 사용할 VAO 불러오기
    glBindVertexArray(vao);

    //--- 삼각형 그리기
    glDrawArrays(GL_TRIANGLES, 0, 3);
}
```

최종 예) 위치, 색상 속성을 사용하여 화면 중앙에 삼각형 그리기

//--- 버텍스 셰이더 객체 만들기

void make_vertexShaders()

```
{
    std::string vertexSource = filetoBuf("vertex.glsl");
    const char* source = vertexSource.c_str();

    //--- 셰이더 생성하고 코드 연결한 후 컴파일 하기
    vertexShader = glCreateShader(GL_VERTEX_SHADER);
    glShaderSource(vertexShader, 1, &source, NULL);
    glCompileShader(vertexShader);

    GLint result;
    GLchar errorLog[512];

    glGetShaderiv(vertexShader, GL_COMPILE_STATUS, &result);

    if (!result)
    {
        glGetShaderInfoLog(vertexShader, 512, NULL, errorLog);
        std::cerr << "ERROR: vertex shader 컴파일 실패\n" << errorLog << std::endl;
        return;
    }
}
```

//--- 프래그먼트 셰이더 객체 만들기

void make_fragmentShaders()

```
{
    std::string fragmentSource = filetoBuf("fragment.glsl");
    const char* source = fragmentSource.c_str();

    //--- 셰이더 생성하고 코드 연결한 후 컴파일 하기
    fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
    glShaderSource(fragmentShader, 1, &source, NULL);
    glCompileShader(fragmentShader);

    GLint result;
    GLchar errorLog[512];

    glGetShaderiv(fragmentShader, GL_COMPILE_STATUS, &result);

    if (!result)
    {
        glGetShaderInfoLog(fragmentShader, 512, NULL, errorLog);
        std::cerr << "ERROR: fragment shader 컴파일 실패\n" << errorLog << std::endl;
        return;
    }
}
```

최종 예) 위치, 색상 속성을 사용하여 화면 중앙에 삼각형 그리기

//--- 버퍼 생성하고 데이터 받아오기

GLuint vao, vbo[2];

void **InitBuffer** ()

```
{
    glGenVertexArrays (1, &vao);           //--- VAO 를 생성하고 ID 얻기
    glBindVertexArray (vao);               //--- VAO를 현재 사용할 대상으로 지정

    glGenBuffers (2, vbo);                 //--- 2개의 VBO를 생성하고 ID할당

    //--- 1번째 VBO를 활성화하여 바인드하고, 버텍스 속성 (좌표값)을 저장
    glBindBuffer(GL_ARRAY_BUFFER, vbo[0]);

    //--- triShape 배열의 사이즈: sizeof(triShape) = 9 * float
    glBufferData(GL_ARRAY_BUFFER, sizeof(triShape), triShape, GL_STATIC_DRAW);

    //--- 좌표값을 attribute 인덱스 0번에 명시한다: 버텍스 당 3* float
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);

    //--- attribute 인덱스 0번을 사용가능하게 함
    glEnableVertexAttribArray(0);

    //--- 2번째 VBO를 활성화 하여 바인드 하고, 버텍스 속성 (색상)을 저장
    glBindBuffer(GL_ARRAY_BUFFER, vbo[1]);

    //--- colors 배열의 사이즈: sizeof(colors) = 9 *float
    glBufferData(GL_ARRAY_BUFFER, sizeof(colors), colors, GL_STATIC_DRAW);

    //--- 색상값을 attribute 인덱스 1번에 명시한다: 버텍스 당 3*float
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, 0);

    //--- attribute 인덱스 1번을 사용 가능하게 함.
    glEnableVertexAttribArray(1);
}
```

//--- 셰이더 프로그램 생성하기

int **make_shaderProgram** ()

```
{
    //--- shader Program
    shaderProgramID = glCreateProgram();

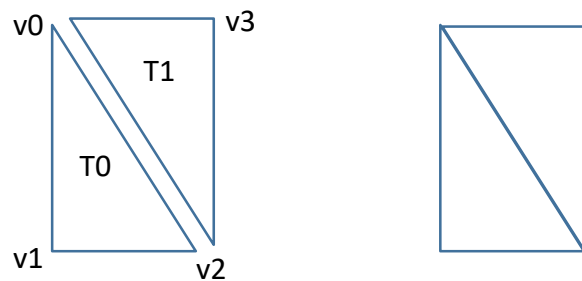
    glAttachShader (shaderProgramID, vertexShader);
    glAttachShader (shaderProgramID, fragmentShader);
    glLinkProgram (shaderProgramID);

    //--- 셰이더 삭제하기
    glDeleteShader (vertexShader);
    glDeleteShader (fragmentShader);

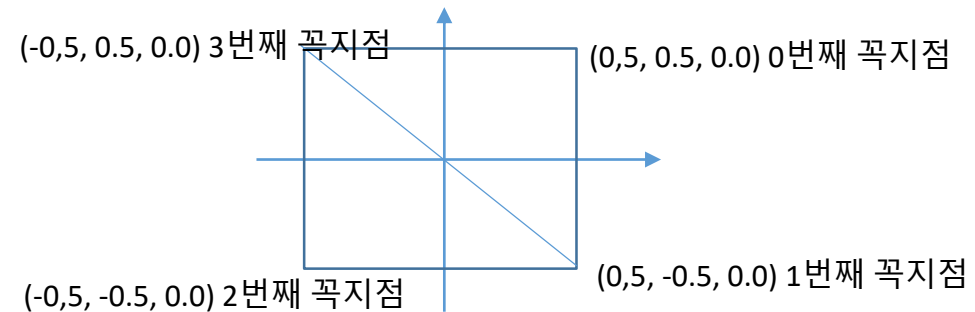
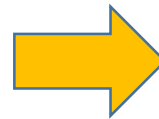
    //--- Shader Program 사용하기
    glUseProgram (shaderProgramID);
    return shaderProgramID;
}
```

Element Buffer Object

- Element Buffer Object (EBO)
 - 정점의 인덱스(index) 정보를 GPU의 버퍼에 저장하는 객체
 - VBO에 저장된 정점들을 어떤 순서로 사용할 것인지를 저장하는 버퍼
 - 정점의 중복을 줄이고 정점을 재사용하기 위해서 사용
 - VBO에는 정점 데이터, EBO는 정점의 사용 순서를 저장한다.
 - 예를 들어, 사각형을 그린다면,
 - 사각형 -> 삼각형 2개 -> 6개의 정점 -> 2개의 정점이 중복, 즉 4개의 정점으로 생성 가능



```
Float vPosition[] = {  
    // 첫 번째 삼각형  
    0.5f, 0.5f, 0.0f, // 우측 상단  
    0.5f, -0.5f, 0.0f, // 우측 하단  
    -0.5f, 0.5f, 0.0f, // 좌측 상단  
    // 두 번째 삼각형  
    0.5f, -0.5f, 0.0f, // 우측 하단  
    -0.5f, -0.5f, 0.0f, // 좌측 하단  
    -0.5f, 0.5f, 0.0f // 좌측 상단  
};
```



```
float vPositionList[] = {  
    0.5f, 0.5f, 0.0f, // 우측 상단  
    0.5f, -0.5f, 0.0f, // 우측 하단  
    -0.5f, -0.5f, 0.0f, // 좌측 하단  
    -0.5f, 0.5f, 0.0f // 좌측 상단  
};  
unsigned int index[] = {  
    0, 1, 3, // 첫 번째 삼각형  
    1, 2, 3 // 두 번째 삼각형  
};
```

Element Buffer Object

- 정점들을 저장하고, 그 정점을 사용하여 인덱스 리스트를 만든다.

```
void InitBuffer ()
{
    GLuint VAO, VBO_pos, EBO;

    glGenVertexArrays (1, &VAO);
    glGenBuffers (1, &VBO_pos);

    glBindVertexArray (VAO);
    glBindBuffer (GL_ARRAY_BUFFER, VBO_pos);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vPositionList), vPositionList, GL_STATIC_DRAW);

    glGenBuffers (1, &EBO);
    glBindBuffer (GL_ELEMENT_ARRAY_BUFFER, EBO);          //--- GL_ELEMENT_ARRAY_BUFFER 버퍼 유형으로 바인딩
    glBufferData (GL_ELEMENT_ARRAY_BUFFER, sizeof(index), index, GL_STATIC_DRAW);
    glVertexAttribPointer (0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), 0);
    glEnableVertexAttribArray (0);
}

Void drawScene ()
{
    glUseProgram (shaderProgramID);
    glBindVertexArray (VAO);
    glDrawElements (GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0);    //--- 0번부터 6개의 꼭지점을 사용하여 삼각형을 그린다.
}
```

함수 프로토타입

- 함수 프로토타입

- void **glDrawElements** (GLenum mode, GLsizei count, GLenum type, const GLvoid *indices);
 - mode: GL_POINTS, GL_LINE_STRIP, GL_LINE_LOOP, GL_LINES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN, GL_TRIANGLES
 - count: 사용할 인덱스의 개수
 - Type: 인덱스 데이터의 자료형 (GL_UNSIGNED_BYTE, GL_UNSIGNED_SHORT, or GL_UNSIGNED_INT 중 1개)
 - indices: 인덱스 데이터의 시작 위치
- void **glBindBuffer** (GLenum target, GLuint buffer);
 - 버퍼 오브젝트를 바인드 한다.
 - Target: 바인드할 버퍼 타겟 타입
 - **GL_ELEMENT_ARRAY_BUFFER**: 버텍스 배열 인덱스
 - Buffer: 버퍼 오브젝트 이름
- void **glBufferData** (GLenum target, GLsizeiptr size, const GLvoid *data, GLenum usage);
 - 버퍼 오브젝트의 데이터를 생성
 - target: 바인드할 버퍼 타겟 타입
 - **GL_ELEMENT_ARRAY_BUFFER**
 - size: 버퍼 오브젝트의 크기
 - data: 저장할 데이터를 가리키는 포인터
 - usage: 저장한 데이터를 사용할 패턴
 - GL_STATIC_DRAW: 한번 버텍스 데이터 업데이트 후 변경이 없는 경우 사용
 - GL_STREAM_DRAW: 데이터가 그려질 때마다 변경
 - GL_DYNAMIC_DRAW: 버텍스 데이터가 자주 바뀌는 경우 (애니메이션), 버텍스 데이터가 바뀔 때마다 다시 업로드 된다

함수 프로토타입

- `glDrawArrays`와 `glDrawElements` 사용 예: 오각형 그리기

```
void DrawScene(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.5f, 0.5f, 0.5f);

    GLfloat vertices[] = {
        0.0f, 0.4f, 0.f,
        -0.4f, 0.1f, 0.f,
        -0.2f, -0.3f, 0.f,
        0.2f, -0.3f, 0.f,
        0.0f, 0.4f, 0.f,
        0.2f, -0.3f, 0.f,
        0.4f, 0.1f, 0.f
    };
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glDrawArrays(GL_TRIANGLES, 2, 3);
    glDrawArrays(GL_TRIANGLES, 4, 3);

    glFlush();
}
```

```
void DrawScene(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.5f, 0.5f, 0.5f);

    GLfloat vertices[] = {
        0.0f, 0.4f, 0.f,
        -0.4f, 0.1f, 0.f,
        -0.2f, -0.3f, 0.f,
        0.2f, -0.3f, 0.f,
        0.4f, 0.1f, 0.f
    };
    GLubyte indices[] = { 0,1,2, 2,3,0, 0,3,4 };

    glDrawElements(GL_TRIANGLES, 9, GL_UNSIGNED_BYTE, indices);

    glFlush();
}
```